IoT AWARENESS PROGRAM

4-Week Online Cohort

Session 4: Environment Sensing — OLED, DHT11 & Buzzer

Student Notes

"Inspire, Learn, Innovate"

mamuzaengineering@gmail.com | 0745 927 033

Welcome to Session 4

In Session 3 you learned to control an LED and read an analog signal from a potentiometer, and you were set an assignment to research the 0.96" OLED display. Today we build on that: you will get the OLED fully working, add a temperature and humidity sensor, add a passive buzzer, and finally combine all three into one working project — a live Environment Monitor with a high-temperature alarm.

Components used this session

- ESP32 development board
- 0.96" OLED display (SSD1306, I2C)
- DHT11 temperature & humidity sensor
- Passive buzzer
- Solderless breadboard and jumper wires

By the end of Session 4, you will be able to:

✓ Explain how the OLED display, the DHT11 sensor, and a passive buzzer each work

✓ Wire and code each component on its own as an individual mini-project

✓ Install and use third-party Arduino libraries correctly

✓ Explain the difference between an active and a passive buzzer, and how tone() works

✓ Explain what the DHT11's 1-wire protocol is and why sampling rate matters

✓ Combine all three components into one master project: a live Environment Monitor with a high-temperature alarm

> ⚠ **A NOTE ON WIRING**
> Wiring tables are included below so you know exactly which ESP32 pin each component pin connects to. Your trainer will also share pinout images for each component — use both together to confirm you're connecting the right physical pin before powering anything on.

Part 1 — The ESP32 as the Central Brain

> 🎯 **Objective:** Understand the ESP32's role in a multi-component project before wiring anything.
>
> ☑ **Expectation:** You can explain, in your own words, why the ESP32 is described as the ‘brain’ of this build.

In every project so far, the ESP32 has been the one constant. Today's build uses three peripherals at once, and the ESP32's job stays exactly the same for each: it processes digital input signals from sensors, runs the control logic you write, and sends data out to peripherals over digital pins and communication buses such as I2C.

The difference today is that these peripherals must all share the ESP32's limited pins sensibly, and your code must manage all three at once — reading a sensor, updating a display, and deciding when to sound an alarm, all inside the same loop().

Part 2 — The 0.96" OLED Display (SSD1306)

🎯 **Objective:** Understand how an OLED screen produces an image and how it communicates with the ESP32 over I2C.

☑ **Expectation:** You can explain what makes an OLED different from an LCD, and identify the SDA/SCL pins on the module.

2.1 How It Works

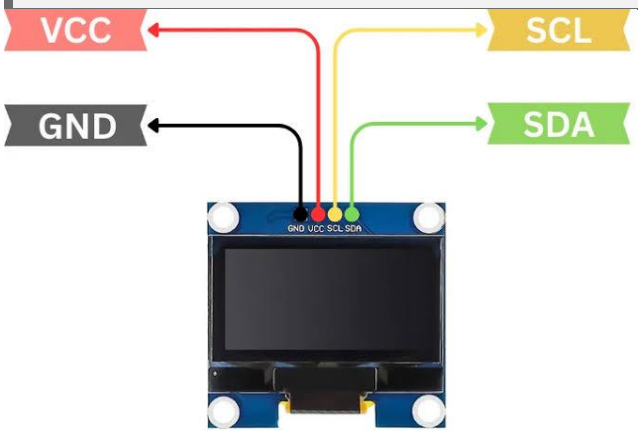
The OLED display is made of self-illuminating pixels — each pixel produces its own light, so unlike an LCD screen, the display needs no backlight. This keeps it thin, sharp, and low-power. Our module has a resolution of 128x64, meaning it is made of a grid 128 pixels wide and 64 pixels tall.

It communicates with the ESP32 using the I2C protocol over just two wires: SCL (the shared clock line) and SDA (the shared data line) — the same protocol you researched for your Session 3 assignment.

2.2 Wiring

OLED Pin	ESP32 Pin	Description
VCC	3.3V / 5V	Power supply
GND	GND	Ground
SCL	GPIO 22	I2C Serial Clock
SDA	GPIO 21	I2C Serial Data

📌 **0.96" OLED (SSD1306) PINOUT IMAGE HERE**



2.3 What is a Library, and How Do You Install One?

🎯 **Objective:** Understand what an Arduino library is and install one correctly using the Library Manager.

☑ **Expectation:** You can install a named library yourself, without step-by-step help, and explain why we use libraries instead of writing everything from scratch.

A library is a pre-written bundle of code that handles a complex task for you — in this case, all the low-level work of sending pixel data to the OLED over I2C. Instead of writing that from scratch, we install a library and simply call its ready-made functions.

To install a library:

1. Open the Arduino IDE and go to Sketch → Include Library → Manage Libraries
2. Type the library name into the search box
3. Find the correct result and click Install

Required Libraries for the OLED

- Adafruit SSD1306
- Adafruit GFX Library

2.4 OLED Mini-Project Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
int counter = 0;

void setup() {
  Serial.begin(115200);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("SSD1306 allocation failed");
    for (;;);   // Stop here if the OLED isn't detected
  }

  display.clearDisplay();
  display.setTextColor(WHITE);
}

void loop() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("OLED Test Project");
  display.drawLine(0, 10, 128, 10, WHITE);

  display.setTextSize(2);
  display.setCursor(0, 25);
  display.print("Count: ");
  display.print(counter++);

  display.display();
  delay(1000);
}
```

What's happening, line by line

- display.begin(SSD1306_SWITCHCAPVCC, 0x3C) — starts the display at I2C address 0x3C, the standard address for this module
- display.clearDisplay() — wipes the internal drawing buffer (not yet the screen itself)
- display.setCursor(x, y) — moves the “writing position” to a pixel coordinate before printing text
- display.drawLine(x0, y0, x1, y1, color) — draws a straight line between two points
- display.display() — pushes everything drawn so far from the buffer onto the physical screen

Part 3 — The DHT11 Temperature & Humidity Sensor

🎯 **Objective:** Understand how the DHT11 measures its surroundings and communicates readings over a single wire.

☑ **Expectation:** You can explain what a 1-wire protocol is and why the sensor can only be read once every couple of seconds.

3.1 How It Works

The DHT11 combines two small sensing elements: a capacitive humidity sensor and a thermistor (a resistor whose resistance changes with temperature) for measuring the surrounding air. A tiny microcontroller built into the DHT11 itself reads these raw analog signals internally, converts them, and outputs a ready-made digital reading.

That digital reading leaves the sensor on a single data line, using a proprietary 1-wire protocol — a timed sequence of digital pulses that encode the temperature and humidity values. This is why we don't use analogRead() for the DHT11 despite it sensing an analog quantity: the conversion has already happened inside the sensor itself.

KEY IDEA
The DHT11 is slow by design — it can only be read at a maximum of about once per second, and in practice we sample it every 2 seconds. Reading it faster than that will return invalid or repeated values.

3.2 Wiring

DHT11 Pin	ESP32 Pin	Description
VCC (+)	3.3V / 5V	Power supply
DATA (OUT)	GPIO 4	Digital data signal
GND (−)	GND	Ground

📌 **DHT11 SENSOR PINOUT IMAGE HERE**

3.3 Required Libraries

- DHT sensor library (by Adafruit)
- Adafruit Unified Sensor

3.4 DHT11 Mini-Project Code

```
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("DHT11 Sensor Test Initialized...");
}

void loop() {
  delay(2000);  // Sampling interval (1Hz max)

  float humidity = dht.readHumidity();
  float tempC = dht.readTemperature();

  if (isnan(humidity) || isnan(tempC)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.print("% | Temperature: ");
  Serial.print(tempC);
  Serial.println(" *C");
}
```

What's happening, line by line

- dht.begin() — initialises communication with the sensor
- dht.readHumidity() / dht.readTemperature() — requests a fresh reading over the 1-wire protocol
- isnan(...) — checks whether a reading came back as “Not a Number”, which happens if the timed pulse sequence was misread
- return — exits loop() early on a failed read, so bad data is never used further down

Part 4 — The Passive Buzzer

Objective: Understand the difference between an active and a passive buzzer, and how tone() generates a pitch.

Expectation: You can explain why a passive buzzer needs a changing signal instead of a simple HIGH, and produce a different note by changing one number.

4.1 How It Works

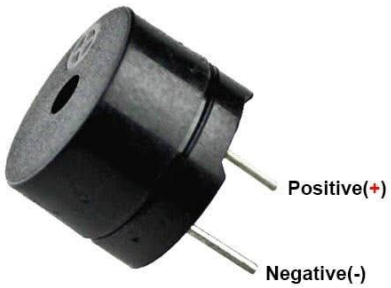
An active buzzer has its own internal oscillator — send it a simple HIGH signal and it plays one fixed tone on its own. A passive buzzer has no oscillator inside it at all: it requires an external alternating Pulse-Width Modulation (PWM) signal from the ESP32 to vibrate its internal piezoelectric disk.

Because we control the alternating signal ourselves, we can change its frequency — and changing the frequency changes the pitch you hear. This means a passive buzzer can play many different tones, beeps, and even simple melodies, while an active buzzer can only ever make one sound.

4.2 Wiring

Buzzer Pin	ESP32 Pin	Description
Positive (+)	GPIO 18	PWM signal output
Negative (−)	GND	Ground

PASSIVE BUZZER PINOUT IMAGE HERE



4.3 Required Libraries

None required — this project uses the tone() function built into the ESP32 Arduino Core.

4.4 Passive Buzzer Mini-Project Code

```
#define BUZZER_PIN 18

void setup() {
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  // Play a sequence of pitch frequencies in Hertz (Hz)
  tone(BUZZER_PIN, 262, 200);  // C4 note
  delay(250);
  tone(BUZZER_PIN, 330, 200);  // E4 note
  delay(250);
  tone(BUZZER_PIN, 392, 200);  // G4 note
  delay(250);
  tone(BUZZER_PIN, 523, 400);  // C5 note
  delay(1000);
}
```

Understanding tone()

tone(pin, frequency, duration) plays a pitch on the given pin at the given frequency (in Hertz) for the given duration (in milliseconds):

Note	Frequency (Hz)
C4	262
E4	330
G4	392
C5	523

Part 5 — Master Project: Environment Monitor & High-Temp Alarm

🎯 **Objective:** Combine the OLED, DHT11, and passive buzzer into one integrated project that senses, displays, and alerts.

☑ **Expectation:** You can explain the full working logic before reading the code, and modify the temperature threshold yourself.

5.1 Working Logic

1. Startup — the ESP32 initialises communication with the OLED display and the DHT11 sensor
2. Reading Cycle — every 2 seconds, current room temperature and humidity are sampled from the DHT11
3. Display Update — the values are formatted and shown live on the OLED screen
4. Alarm Check — if temperature exceeds 30.0°C, the status changes to ALERT! and the passive buzzer plays an alternating alarm tone; otherwise the display reads STATUS: NORMAL and the buzzer stays silent

5.2 Complete Wiring Table

Peripheral	Component Pin	ESP32 Pin
OLED (SSD1306)	SCL	GPIO 22
OLED (SSD1306)	SDA	GPIO 21
OLED (SSD1306)	VCC	3.3V
OLED (SSD1306)	GND	GND
DHT11 Sensor	DATA	GPIO 4
DHT11 Sensor	VCC	3.3V
DHT11 Sensor	GND	GND
Passive Buzzer	Signal (+)	GPIO 18
Passive Buzzer	Ground (−)	GND

5.3 Complete Master Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "DHT.h"

// Screen configuration
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// DHT11 Pin configuration
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// Buzzer & Alarm Threshold
#define BUZZER_PIN 18
#define TEMP_THRESHOLD 30.0  // Threshold in Celsius

void soundAlarm() {
  tone(BUZZER_PIN, 1000, 150);
  delay(150);
  tone(BUZZER_PIN, 1600, 150);
  delay(150);
}

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  dht.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED allocation failed!");
    for (;;);
  }

  // Display Splash Screen
  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.setCursor(10, 25);
  display.println("Initializing System...");
  display.display();
  delay(2000);
}

void loop() {
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();
  display.clearDisplay();

  if (isnan(temp) || isnan(hum)) {
    display.setTextSize(1);
    display.setCursor(0, 20);
    display.println("Sensor Read Error!");
    display.display();
    delay(2000);
    return;
```

```
  }

  // Header Title
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("ENVIRONMENT MONITOR");
  display.drawLine(0, 10, 128, 10, WHITE);

  // Temperature Row
  display.setCursor(0, 16);
  display.print("Temp: ");
  display.setTextSize(2);
  display.print(temp, 1);
  display.setTextSize(1);
  display.println(" C");

  // Humidity Row
  display.setCursor(0, 36);
  display.print("Humidity: ");
  display.setTextSize(2);
  display.print(hum, 1);
  display.setTextSize(1);
  display.println(" %");

  // Status Bar & Threshold Trigger
  display.drawLine(0, 52, 128, 52, WHITE);
  display.setCursor(0, 55);

  if (temp >= TEMP_THRESHOLD) {
    display.print("STATUS: HIGH TEMP!");
    display.display();
    soundAlarm();
  } else {
    display.print("STATUS: NORMAL");
    display.display();
    delay(2000);
  }
}
```

5.4 Code Walkthrough

- Three separate objects (display, dht, and the BUZZER_PIN constant) are set up once, at the top of the file, so the rest of the code can refer to them by name
- setup() shows a short “Initializing System...” splash screen so you get visual confirmation the OLED is alive before the main loop starts
- Each pass through loop() takes one fresh temperature and humidity reading, and immediately redraws the whole screen from scratch
- display.setTextSize(2) is used for the numeric readings and switched back to setTextSize(1) for labels — mixing sizes like this is how you build a readable dashboard layout
- The if (temp >= TEMP_THRESHOLD) check is the single decision point that drives the entire alarm behaviour — change TEMP_THRESHOLD and the whole system's sensitivity changes with it
- soundAlarm() plays two alternating tones once; because there's no delay(2000) in the alarm branch, the loop repeats quickly while temperature stays high, creating a continuous alternating siren

5.5 Test & Troubleshoot

Symptom	Likely Cause
"SSD1306 allocation failed" / "OLED allocation failed!"	OLED not wired correctly, or wrong I2C address (confirm it's 0x3C)
"Failed to read from DHT sensor!" repeatedly	DHT11 DATA pin miswired, or reading faster than once every 2 seconds
Buzzer silent even during an alert	Buzzer wired to the wrong GPIO, or a passive buzzer confused for an active one
Display shows numbers that never change	DHT11 GND or VCC not connected, giving a stuck/failed reading

CHECKPOINT

If your OLED shows live temperature and humidity, and the buzzer sounds the moment you warm the DHT11 above 30°C (try cupping it in your hand), your Environment Monitor is fully working — you've just built your first multi-sensor system!

Session 4 Recap

Today you moved from single-component projects to a full multi-sensor system. Here's what you now know:

- ✓ How an OLED display produces an image and communicates over I2C
- ✓ How to install and use third-party Arduino libraries
- ✓ How the DHT11 measures temperature and humidity and reports it over a 1-wire protocol
- ✓ The difference between an active and a passive buzzer, and how tone() controls pitch
- ✓ How to combine multiple peripherals into one coordinated project using a shared loop()
- ✓ How to design and adjust a simple threshold-based alarm system

Looking Ahead to Session 5

Next session we expand your sensing toolkit further — you'll add soil moisture and rain sensors, moving one step closer to the smart-irrigation capstone project.